

Session 3: Lists, loops, and decisions

Many values under one name,
work that repeats, and programs
that choose.

Data Science and Advanced Programming · UNIL ·
2026

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]  ← Three positions, one name
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

Session 1, again: three positions

In session 1's workshop you valued the ledger like this:

```
aapl = 10 * 190.5
nesn = 4 * 102.3
voo = 2 * 512.0
print(aapl + nesn + voo)
```

3338.2

Fine for three positions. The real ledger has 300.
Tomorrow it has 301.

Today: data that comes as a **collection**, and code that **repeats** and **decides**.

```
$ python ledger.py
not run yet
```

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

Today: one script, the whole ledger

1. Lists: order, indexing, slicing
2. `for` loops: repeat for every item
3. Decisions: `if` / `elif` / `else`
4. Dictionaries: look up by name
5. `while` loops: repeat until done
6. Libraries bring their own types: numpy and pandas

Running through all of it: **every type comes with its own operations.**

```
$ python ledger.py
```

```
not run yet
```

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
not run yet
```

Read it cold

Two minutes, on your own.

- Which lines can you already read?
- Which symbols are new? (Look for `[ ]`, `{ }`, `for`, `if`, `while`.)
- **Guess the output.** How many lines will it print?

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"] <The data>
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0 <Value every position>
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0 <How long to double?>
20  grown = total
21  while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

The shape of the script

Lines	Job
2–5	the data: a list, another list, a dict
7–17	a loop: value every position, and label its size
19–24	another loop: how many years until the total doubles

Three lines of data, but line 17 prints **three** times. That is a loop.

```
$ python ledger.py
not run yet
```

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

What it prints

You do not need to run anything now. Bottom left is the output of one run of the script.

Five lines of output from 24 lines of code. Lines 2–4 print once, line 17 prints three times, line 24 once.

By the end of today you can say exactly why.

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
```

```
AAPL 1,905.00 large
```

one line per position

```
NESN 409.20 small
```

```
VOO 1,024.00 medium
```

```
total 3,338.20, doubles in 15 years
```

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]  ← A new type: list
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[-1], tickers[1:], price["NESN"])  ← Another: dict
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

REMINDER

Every value has a type (session 2)

`str`, `int`, `float`, `bool`, `None`: each value has exactly one type, and `type()` tells you which.

Today adds two types that **hold other values**:

```
tickers = ["AAPL", "NESN", "VOO"]
price = {"AAPL": 190.5}
print(type(tickers), type(price))
```

```
<class 'list'> <class 'dict'>
```

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value  # Here: numbers, so add
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

REMINDER

A type comes with its own operations (session 2)

The same symbol means a different operation for each type:

You write	You get	What <code>+</code> means
<code>3 + 4</code>	<code>7</code>	<code>int</code> : add
<code>"3" + "4"</code>	<code>"34"</code>	<code>str</code> : join
<code>3 + "4"</code>	<code>TypeError</code>	no operation for this pair

Today's new types bring their own `+`, `*`, `[ ]` and more. Keep asking: **what type is this, and what does this operator do to it?**

PART 1

Lists

Many values, in order, under one name.

```

1  # ledger.py
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers, len(tickers), shares, len(shares))
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")

```

```

$ python ledger.py
['AAPL', 'NESN', 'VOO'] 3 [10, 4, 2] 3
AAPL    1,905.00  large
NESN     409.20  small
VOO     1,024.00  medium
total 3,338.20, doubles in 15 years

```

A list: many values, one name

Square brackets, items separated by commas. A list is **one value** of type `list`, and it keeps its items **in order**.

Line 5 now prints both lists and their lengths. `len()` (session 2, for text) also counts the items of a list.

ledger.py

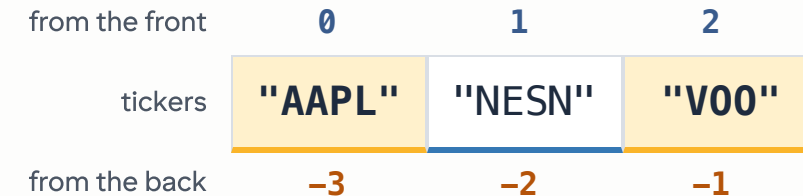
```
1 # ledger.py - the whole ledger: lists, a dict, loops
2 tickers = ["AAPL", "NESN", "VOO"]
3 shares = [10, 4, 2]
4 price = {"AAPL": 102.3, "NESN": 102.3, "VOO": 512.0}
5 print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7 total = 0.0
8 for i in range(len(tickers)):
9     value = shares[i] * price[tickers[i]]
10    total += value
11    if value >= 1500:
12        size = "large"
13    elif value >= 500:
14        size = "medium"
15    else:
16        size = "small"
17    print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19 years = 0
20 grown = total
21 while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24 print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL 1,905.00 large
NESN 409.20 small
VOO 1,024.00 medium
total 3,338.20, doubles in 15 years
```

Indexing: positions start at 0

`tickers[0]` is the **first** item. Count from the front with 0, 1, 2; from the back with -1, -2, -3:



Why 0? The index says **how many steps from the start**. The first item is zero steps away.

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[3], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")

```

Index 3 does not exist

One step too far: IndexError

On the left, line 5 now asks for `tickers[3]`. Three items have indices **0, 1, 2**. There is no 3, and the program stops on line 5.

The classic mistake: counting from 1 in your head. `tickers[len(tickers)]` is always one too far. The last item is `tickers[len(tickers) - 1]`, or simply `tickers[-1]`.

```
$ python ledger.py
```

```
Traceback (most recent call last):
```

```
File "ledger.py", line 5, in <module>
```

```
    print(tickers[3], tickers[-1], tickers[1:], price["NESN"])
```

```
~~~~~^^^
```

```
IndexError: list index out of range
```

ledger.py

```
1 # ledger.py - the whole ledger: lists, a dict, loops
2 tickers = ["AAPL", "NESN", "VOO"]
3 shares = [10, 4, 2]
4 price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5 print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7 total = 0.0
8 for i in range(len(tickers)):
9     value = shares[i] * price[tickers[i]]
10    total += value
11    if value >= 1500:
12        size = "large"
13    elif value >= 500:
14        size = "medium"
15    else:
16        size = "small"
17    print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19 years = 0
20 grown = total
21 while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24 print(f"total {total:.,.2f}, doubles in {years} years")
```

From 1 to the end

Slicing: a piece of a list

`tickers[start:stop]` takes the items **from start up to, but not including, stop**. Think of the numbers as cuts between the boxes:



```
tickers = ["AAPL", "NESN", "VOO"]
print(tickers[0:2], tickers[:2], tickers[1:], tickers[:])
```

```
['AAPL', 'NESN'] ['AAPL', 'NESN'] ['NESN', 'VOO']
['AAPL', 'NESN', 'VOO']
```

Leave out `start` : from the beginning. Leave out `stop` : to the end.

\$ python ledger.py

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL 1,905.00 large
NESN 409.20 small
VOO 1,024.00 medium
total 3,338.20, doubles in 15 years
```

A list again

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[1:10], tickers[5:], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")

```

Past the end

\$ python ledger.py

['NESN', 'VOO'] [] ['NESN', 'VOO'] 102.3

Less, not a crash

```

AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years

```

Slices never crash

An index past the end crashes. A slice past the end just gives **less**: line 5 now slices to 10, and from 5.

`[]` is the **empty list**: a list with no items. `len([])` is `0`.

So `tickers[3]` crashes, but `tickers[3:]` quietly returns `[]`. Know which one you wrote.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL    1,905.00  large
NESN     409.20  small
VOO     1,024.00  medium
total 3,338.20, doubles in 15 years
```

+ GOOD TO KNOW

Slicing with a step

A third number in the slice is the **step**:

```
tickers = ["AAPL", "NESN", "VOO"]
print(tickers[::2])
print(tickers[::-1])
print(list(range(0, 10, 3)))
```

```
['AAPL', 'VOO']
['VOO', 'NESN', 'AAPL']
[0, 3, 6, 9]
```

`[::2]`: every second item. `[::-1]`: the list backwards, a common trick. `range` takes the same three numbers: start, stop, step.

slices.py

```
1  # slices.py - slicing, in many types
2  import numpy as np
3  import pandas as pd
4
5  ticker = "NESN.SW"
6  print(ticker[:4], ticker[-2:])  text
7
8  prices = [190.5, 102.3, 512.0, 410.0]
9  print(prices[1:3], prices[::-1])  a list
10
11 table = np.array([[10, 190.5], [4, 102.3], [2, 512.0]])
12 print(table[1:, 0], table[:2, 1])  a table of numbers
13
14 names = ["AAPL", "NESN", "VOO"]
15 ledger = pd.DataFrame({"shares": [10, 4, 2]}, index=names)
16 print(ledger.iloc[:2]["shares"].tolist())  a pandas table
17 print(ledger.loc["AAPL":"NESN", "shares"].tolist())
18
19 parts = "NESN,4,102.30".split(",")
20 print(parts[1:])  the pieces of a line
```

```
$ python slices.py
```

```
NESN SW
[102.3, 512.0] [410.0, 512.0, 102.3, 190.5]
[4. 2.] [190.5 102.3]
[10, 4]
[10, 4]
['4', '102.30']
```

⚠ WARNING

Slicing looks simple. It will not stay simple.

On the left: **six kinds of objects**, all sliced with `[ ]`. You do not need to know what they are yet. Look only at the square brackets.

In data science you will slice **all the time**, and each type decides what its slice means: some slice in two directions at once, some slice by name, some include the stop.

When a slice surprises you, check the **type** first.

ledger.py

```
1 # ledger.py - the whole ledger: lists, a dict, loops
2 tickers = ["AAPL", "NESN", "VOO"] Text is a sequence too
3 shares = [10, 4, 2]
4 price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5 print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7 total = 0.0
8 for i in range(len(tickers)):
9     value = shares[i] * price[tickers[i]]
10    total += value
11    if value >= 1500:
12        size = "large"
13    elif value >= 500:
14        size = "medium"
15    else:
16        size = "small"
17    print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19 years = 0
20 grown = total
21 while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24 print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

Strings index and slice the same way

A `str` is a sequence of characters. `[ ]` works on it just like on a list:

```
ticker = "NESN"
print(ticker[0], ticker[-1], ticker[1:3], len(ticker))
```

```
N N ES 4
```

One rule for both: **positions from 0, stop not included.**

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  parts = "NESN,4,102.30".split(",")  < Cut at every comma
6  print(parts, int(parts[1]) * float(parts[2]))
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")

```

```
$ python ledger.py
```

```
['NESN', '4', '102.30'] 409.2 < A list of text
```

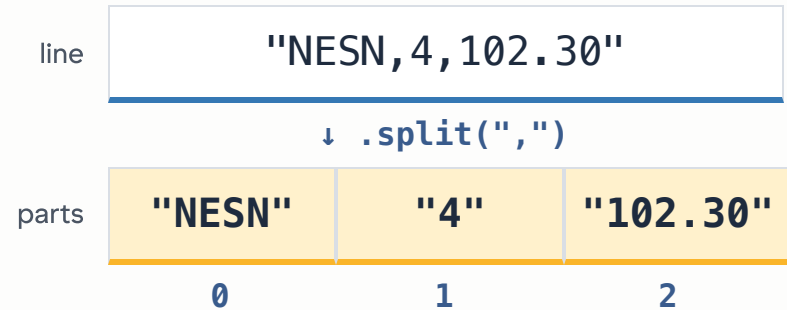
```

AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years

```

Text becomes a list: .split()

A line of `ledger.csv` is one string. `.split(",")` cuts it at every comma:



Lines 5–6 on the left do it for real. Every piece is still **text**: convert before you compute (session 2). Session 6 reads the whole file this way.

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  shares[1] = 6  ← Replace item 1
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")

```

```
$ python ledger.py
```

```

AAPL  1,905.00  large
NESN   613.80  medium  ← NESN: now 6 shares
VOO    1,024.00  medium
total 3,542.80, doubles in 15 years

```

Lists can change. Strings cannot.

Line 5 now **replaces** item 1 of `shares`: NESN has 6 shares, and the loop values it at 613.80.

`shares.append(3)` adds an item at the end. `.append` is a **method**: an operation that belongs to the `list` type, written after a dot.

```

ticker = "NESN"
ticker[0] = "M"

```

```

TypeError: 'str' object does not support item
assignment

```

Same `[ ]`, different type, different rules.

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(shares + [7], shares * 2, 4 in shares,
6        sum(shares), max(shares))
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")

```

\$ python ledger.py

[10, 4, 2, 7] [10, 4, 2, 10, 4, 2] True 16 10 + joins

AAPL 1,905.00 large
 NESN 409.20 small
 VOO 1,024.00 medium

total 3,338.20, doubles in 15 years

The list's own operations

You write	What it does
<code>a + b</code>	joins two lists into a new one
<code>a * 2</code>	repeats the list
<code>x in a</code>	True if <code>x</code> is an item
<code>len(a)</code> , <code>sum(a)</code> , <code>min(a)</code> , <code>max(a)</code>	count, add up, smallest, largest
<code>a.append(x)</code>	adds <code>x</code> at the end

Lines 5–6 try five of them on `shares`. `+` on lists **joins**. It never adds the numbers inside. Remember this when we meet numpy.

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 409.2, "VOO": 1024}
5  print(shares + [7], shares * 2, 4 in shares,
6        sum(shares), max(shares))
7  total = 0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")

```

```
$ python ledger.py
```

```

[10, 4, 2, 7] [10, 4, 2, 10, 4, 2] True 16 10
AAPL  1,905.00  large
NESN   409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years

```

Did you notice? One statement, two lines

On the previous slide, line 5 was too long for the screen, so it continues on line 6. Python still reads it as **one** statement: one `print`, one output line.

The rule: **inside brackets** `()`, `[]` or `{}`, you may break a line anywhere after a comma. Python waits for the closing bracket.

- Indent the continuation so it lines up; it is only for the reader.
- Keep lines short, about 80 characters (PEP 8, session 2).
- Outside brackets, a break ends the statement.

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(sorted(shares), shares)  A new, sorted list
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")

```

Order: sorted() makes a new list

A list keeps the order you gave it. Sorting is something you ask for.

On the left, line 5 prints `sorted(shares)` and then `shares`. `sorted()` returns a **new**, sorted list; `shares` itself is unchanged.

```
$ python ledger.py
```

```

[2, 4, 10] [10, 4, 2]  shares: unchanged
AAPL  1,905.00  large
NESN   409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years

```

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  shares.sort()  < Sorts shares itself
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")

```

.sort() changes the list itself

Line 5 now calls `shares.sort()`. It sorts **this** list, in place: `shares` becomes `[2, 4, 10]`.

The program runs, but look at the output: AAPL is valued with **2** shares (381.00) and VOO with **10**. Our `tickers` and `shares` must stay in the **same** order; sort one alone and they no longer match.

`sorted()` gives you a new list and leaves the old one alone. `.sort()` changes the old one.

```
$ python ledger.py
```

```

AAPL    381.00  small  < AAPL valued with 2 shares!
NESN    409.20  small
VOO     5,120.00 large
total 5,910.20, doubles in 15 years

```

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  backup = shares
6  backup[1] = 6
7
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")

```

Not a copy: a second name

Changes backup ...

```
$ python ledger.py
```

```

AAPL  1,905.00  large
NESN   613.80  medium
VOO    1,024.00  medium
total 3,542.80, doubles in 15 years

```

... and shares too!

⚠ WARNING

Two names, one list

Line 5 looks like session 2's assignment: `backup` gets the value of `shares`. Line 6 changes `backup` only. Yet the loop, which reads `shares`, now values NESN at 613.80: `shares` **changed too**.

`backup = shares` does **not** copy the list. It puts a second name on the **same** list. A change through one name shows through the other.

- For an independent copy: `backup = shares.copy()`.
- Numbers and text cannot change, so they never surprise you like this. Lists, dicts and numpy arrays can.

PART 2

for loops

Do the same work for every item. Write it once.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

Repeat for each item

```
tickers = ["AAPL", "NESN", "VOO"]
for ticker in tickers:
    print("valuing", ticker)
print("done")
```

```
valuing AAPL
valuing NESN
valuing VOO
done
```

The indented line runs **once per item**. Each time, `ticker` holds the next item, in order.

`print("done")` is not indented: it runs once, **after** the loop.

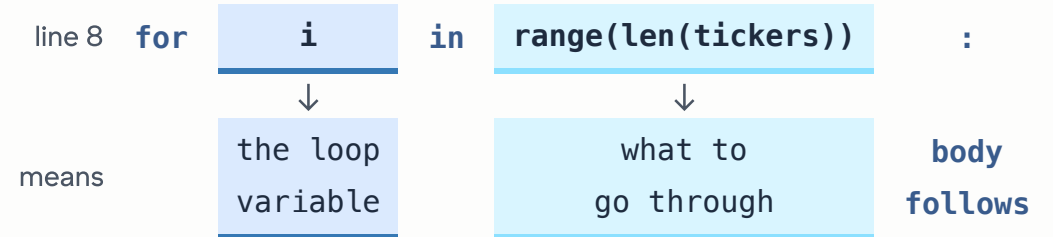
ledger.py

```
1 # ledger.py - the whole ledger: lists, a dict, loops
2 tickers = ["AAPL", "NESN", "VOO"]
3 shares = [10, 4, 2]
4 price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5 print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7
8 for i in range(len(tickers)):
9     value = shares[i] * price[tickers[i]]
10    total += value
11    if value >= 1500:
12        size = "large"
13    elif value >= 500:
14        size = "medium"
15    else:
16        size = "small"
17    print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19 years = 0
20 grown = total
21 while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24 print(f"total {total:.,.2f}, doubles in {years} years")
```

The loop header

for **i** **in** **range(len(tickers))** **:**

Anatomy of a for loop: line 8



- The **body** is every indented line after the **:**, here lines 9–17.
- Python runs the body once for each value, with **i** set to that value.
- When the values run out, Python continues after the body (line 19).

```
$ python ledger.py
```

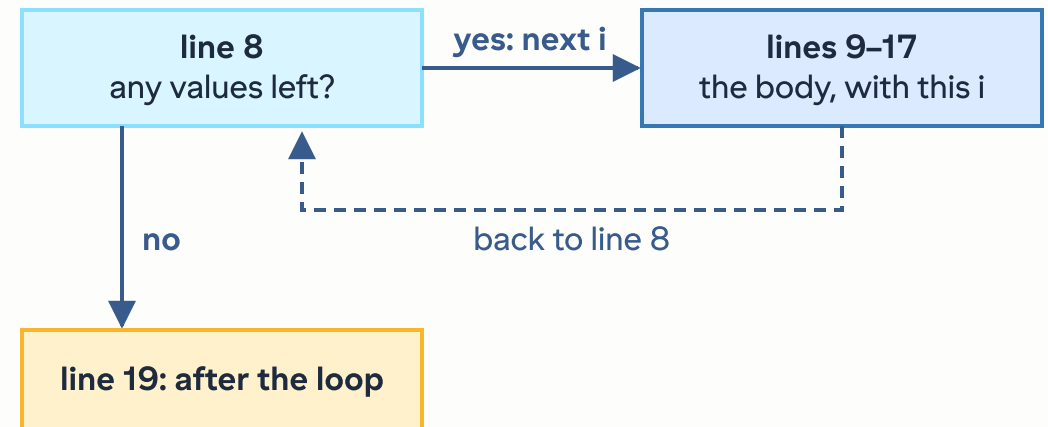
```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL 1,905.00 large
NESN 409.20 small
VOO 1,024.00 medium
total 3,338.20, doubles in 15 years
```

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  Any values left?
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

A loop, as a picture



The loop goes round **three** times, once per position, then leaves by the "no" arrow. Every loop, `for` or `while`, has this shape.

ledger.py

```
1 # ledger.py - the whole ledger: lists, a dict, loops
2 tickers = ["AAPL", "NESN", "VOO"]
3 shares = [10, 4, 2]
4 price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5 print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7 total = 0.0
8 for i in range(len(tickers)): range(3): 0, 1, 2
9     value = shares[i] * price[tickers[i]]
10    total += value
11    if value >= 1500:
12        size = "large"
13    elif value >= 500:
14        size = "medium"
15    else:
16        size = "small"
17    print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19 years = 0
20 grown = total
21 while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24 print(f"total {total:,.2f}, doubles in {years} years")
```

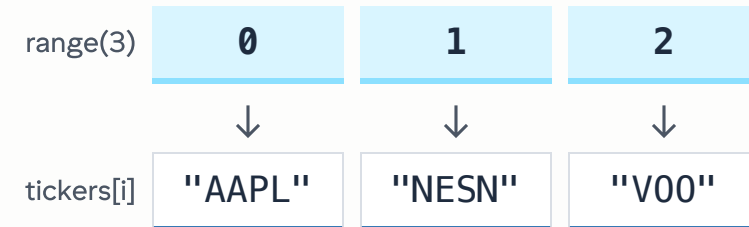
Same i, two lists

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL 1,905.00 large
NESN 409.20 small
VOO 1,024.00 medium
total 3,338.20, doubles in 15 years
```

range(): the numbers a loop counts with

You will use `range` **all the time**: it is how a loop counts.
`range(3)` gives 0, 1, 2, exactly the positions of a 3-item list:



You write	Gives	Like slicing
<code>range(3)</code>	0, 1, 2	stop not included
<code>range(1, 4)</code>	1, 2, 3	start, stop
<code>range(0, 10, 3)</code>	0, 3, 6, 9	start, stop, step

`range(len(tickers))` fits a list of **any** length.

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for ticker in tickers:
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")

```

Items, not positions

Still needs i

for over items, or over positions?

On the left, line 8 now loops over the **items**: `ticker` is "AAPL", then "NESN", then "VOO".

That is the simplest loop, but line 9 still asks for `shares[i]`, and there is no `i` any more: `NameError`.

- `for ticker in tickers`: you get the **items**. Use it when one list is enough.
- `for i in range(len(tickers))`: you get the **positions**. Our script needs them: line 9 uses the same position in two lists.

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
```

```
Traceback (most recent call last):
```

```
File "ledger.py", line 9, in <module>
```

```
    value = shares[i] * price[tickers[i]]
```

```
        ^
```

```
NameError: name 'i' is not defined. Did you mean: 'id'?
```

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0  ← Before the loop
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value  ← Once per pass
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

Tracing the loop

Session 1's tracing table, one row per pass:

pass	i	tickers[i]	value	total after line 10
before	—	—	—	0.0
1	0	"AAPL"	1905.0	1905.0
2	1	"NESN"	409.2	2314.2
3	2	"VOO"	1024.0	3338.2

After the third pass there are no more values. The loop ends, and `total` holds the sum.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0  1. Start at zero
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value  2. Add each item
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
```

```
AAPL 1,905.00 large
```

```
NESN 409.20 small
```

```
VOO 1,024.00 medium
```

```
total 3,338.20, doubles in 15 years 3. Read it after
```

Pattern: the accumulator

Three steps, always the same shape:

1. **Before** the loop, start the total at `0` (line 7).
2. **Inside** the loop, add each item (line 10).
3. **After** the loop, use the result (line 24).

You will write this pattern hundreds of times: totals, counts, averages.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

Pattern: count and collect

The same shape, with a condition inside:

```
tickers = ["AAPL", "NESN", "VOO"]
values = [1905.0, 409.2, 1024.0]
count = 0
large = []
for i in range(len(values)):
    if values[i] >= 1000:
        count += 1
        large.append(tickers[i])
print(count, large)
```

```
2 ['AAPL', 'VOO']
```

Counting: start at `0`, add `1`. Collecting: start with `[]`,
`.append` each match.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

Pattern: find the largest

Keep the best so far, and replace it when you meet something bigger:

```
values = [1905.0, 409.2, 1024.0]
best = values[0]
for v in values:
    if v > best:
        best = v
print(best, max(values))
```

```
1905.0 1905.0
```

Python's `max()` does the same in one call. Write the loop once to see how it works; then use `max()`.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

A loop inside a loop

The inner loop runs completely, **for every pass** of the outer loop:

```
tickers = ["AAPL", "NESN", "VOO"]
passes = 0
for ticker in tickers:
    for year in [1, 2]:
        passes += 1
print(passes)
```

6

3 tickers × 2 years = **6** passes of the inner body. Our `if` chain inside the `for` is the same idea: a block inside a block.

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         No longer indented small"
17 print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19 years = 0
20 grown = total
21 while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24 print(f"total {total:.,.2f}, doubles in {years} years")

```

```
$ python ledger.py
```

```

AAPL VOO ['NESN', 'VOO'] 102.3
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years

```

⚠ WARNING

Indentation is part of the syntax

In most languages, indentation is only for the reader. In **Python it is grammar**: the indent decides which block a line belongs to, and so **how the program runs**.

On the left, line 17 has lost four spaces. It left the loop body, so it runs **once**, after the loop, with the values of the last pass: only VOO is printed.

No error. Same words, different program. Session 2 said blocks are indentation; here it changes the result.

Rule: indent with **4 spaces**, and let the editor do it.

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")

```

No colon

Forgot the colon

Line 11 lost its `:`. Look at the output: **not even line 5 ran**. Python reads the whole file before running anything, so a grammar mistake stops everything (session 1: **it won't start**).

Every `for`, `while`, `if`, `elif` and `else` line ends with `:`.

```
$ python ledger.py
```

```
File "ledger.py", line 11
    if value >= 1500
        ^
```

```
SyntaxError: expected ':'
```

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12     size = "large"
13
14     size = "medium"
15 else:
16     size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19 years = 0
20 grown = total
21 while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24 print(f"total {total:,.2f}, doubles in {years} years")

```

Not indented under the if

```
$ python ledger.py
```

```
File "ledger.py", line 12
```

```
size = "large"
```

```
^^^^
```

```
IndentationError: expected an indented block after 'if' statement
on line 11
```

Forgot the indent

Now line 12 is not indented under the `if` on line 11. The `if` has an empty block, and Python refuses to start.

After every `:`, the block must be indented. Editors do it for you when you press Enter after a `:`.

PART 3

Decisions: if, elif, else

Let the program choose which lines run.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:  A bool (session 2)
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

if: run a block only when a condition holds

```
value = 409.2
if value >= 1500:
    print("large")
print("done")
```

```
done
```

`value >= 1500` is a `bool` (session 2). If it is `True`, the indented block runs. If `False`, Python skips it.

Here it is `False`: only `done` is printed.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:  # else if: the next check
14         size = "medium"
15     else:  # none of the above
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

elif and else: more than one way

- `if`: check the first condition.
- `elif` ("else if"): only if everything above was `False`, check this one.
- `else`: only if **everything** above was `False`. No condition.

Exactly one block of the chain runs: the first one whose condition is `True`, or `else`.

Output lines 2–4: one label per position, never two.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00 large
NESN   409.20 small
VOO   1,024.00 medium VOO: 1024.0
total 3,338.20, doubles in 15 years
```

The chain, step by step: VOO

For VOO, value is 1024.0:

line 11	<code>1024.0 >= 1500</code>	→	False
line 13	<code>1024.0 >= 500</code>	→	True
line 14	<code>size = "medium"</code>	← runs	
line 15	<code>else: skipped</code>		

The first `True` wins. Nothing after it in the chain is checked.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

Blocks inside blocks

Lines 8–17 have two levels. The `if` chain sits **inside** the loop, so the whole chain runs once per position:

```
for i in range(len(tickers)):
    value = ... total += value
    if value >= 1500:
        size = "large"
    elif value >= 500:
        size = "medium"
    else:
        size = "small"
    print(...)
```

Blue: runs once per pass. Gold: runs only when its condition is chosen. Read the indentation first; it shows you the structure before the words do.

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 500:
12         size = "medium"
13     elif value >= 1500:
14         size = "large"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")

```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
```

```
AAPL 1,905.00 medium AAPL: 1,905 is not medium!
```

```
NESN 409.20 small
```

```
VOO 1,024.00 medium
```

```
total 3,338.20, doubles in 15 years
```

The order of the checks matters

On the left, the two checks are swapped. 1905 is also `>= 500`, so the first check wins and `large` is never reached: AAPL is labelled medium.

No error. A wrong label, forever.

Rule: in a chain of thresholds, test the **biggest** first.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

Boundaries: >= or >?

A position worth exactly 1500:

```
value = 1500.0
print(value >= 1500, value > 1500)
```

```
True False
```

With `>=` it is large. With `>` it is medium. Both run; both look fine.

Which one is right? **The brief decides**, not the code. When a rule says "above 1500", ask: does 1500 itself count?

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

+ GOOD TO KNOW

Empty means False

`if` also accepts values that are not `bool`. Python asks: is it empty, or zero?

```
print(bool([]), bool(["AAPL"]), bool(0), bool(""))
```

```
False True False False
```

So `if large:` means "if the list is not empty". You will see this often in other people's code, and in AI-written code.

Our course writes the full question: `if len(large) > 0:`. Slower to type, faster to read.

PART 4

Dictionaries

Look a value up by its name, not by its position.

ledger.py

```
1 # ledger.py - the whole ledger: lists, a dict, loops
2 tickers = ["AAPL", "NESN", "VOO"]
3 shares = [10, 4, 2]
4 price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5 print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7 total = 0.0
8 for i in range(len(tickers)):
9     value = shares[i] * price[tickers[i]]
10    total += value
11    if value >= 1500:
12        size = "large"
13    elif value >= 500:
14        size = "medium"
15    else:
16        size = "small"
17    print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19 years = 0
20 grown = total
21 while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24 print(f"total {total:.,.2f}, doubles in {years} years")
```

Look up by key

A dict maps keys to values

Curly braces, and `key: value` pairs:

key		value
"AAPL"	→	190.5
"NESN"	→	102.3
"VOO"	→	512.0

`price["NESN"]` gives 102.3. Same square brackets as a list, but inside goes a **key**, not a position.

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3 price["NESN"]
AAPL 1,905.00 large
NESN 409.20 small
VOO 1,024.00 medium
total 3,338.20, doubles in 15 years
```

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]  # By position: 0, 1, 2
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  # By name: tickers[0], tickers[-1], tickers[1:], price["NESN"]
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

List or dict?

You need...	Use	Look up with
items in order, by position	list	shares[1]
a value for each name	dict	price["NESN"]

Our script has both. `shares` works only because it is in the same order as `tickers`. `price` needs no order at all: every price is attached to its name.

Rule of thumb: if you find yourself writing "the position of NESN", you wanted a dict.

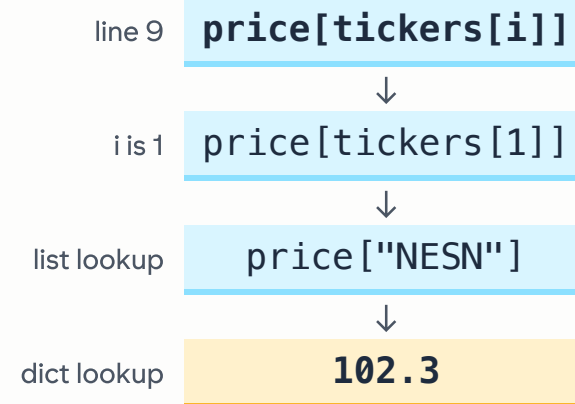
ledger.py

```
1 # ledger.py - the whole ledger: lists, a dict, loops
2 tickers = ["AAPL", "NESN", "VOO"]
3 shares = [10, 4, 2]
4 price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5 print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7 total = 0.0
8 for i in range(len(tickers)):
9     value = shares[i] * price[tickers[i]]
10    total += value
11    if value >= 1500:
12        size = "large"
13    elif value >= 500:
14        size = "medium"
15    else:
16        size = "small"
17    print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19 years = 0
20 grown = total
21 while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24 print(f"total {total:,.2f}, doubles in {years} years")
```

Read it inside out

Reading line 9 inside out

`price[tickers[i]]` has brackets inside brackets. Start with the **innermost**. In the second pass, `i` is `1`:



The list turns a **position** into a **name**. The dict turns the **name** into a **price**.

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["MSFT"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")

```

No MSFT in price

A missing key: KeyError

Line 5 now looks up `"MSFT"`, which is not a key of `price`. The program stops with a `KeyError`, which names the missing key.

A list complains about a **position** (`IndexError`); a dict complains about a **key** (`KeyError`).

```
$ python ledger.py
```

```
Traceback (most recent call last):
```

```
File "ledger.py", line 5, in <module>
```

```
    print(tickers[0], tickers[-1], tickers[1:], price["MSFT"])
                                ~~~~~^
```

```
KeyError: 'MSFT'
```

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print("MSFT" in price, price.get("MSFT", 0.0))
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")

```

```
$ python ledger.py
```

```
False 0.0
```

No crash

```

AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years

```

Ask first: in and .get()

Two safe ways to handle a key that may be missing, on line 5:

- `"MSFT" in price` checks the **keys**: `False`. Like `==`, `in` is an operator that asks a yes/no question, so its answer is a `bool`: you can use it directly in an `if`.
- `price.get("MSFT", 0.0)` returns the default instead of crashing: `0.0`.

Is 0.0 a sensible price? **The brief decides.** Sometimes a crash is the honest answer.

ledger.py CHANGED ON THIS SLIDE

```
1 # ledger.py - the whole ledger: 11 lines, loops
2 tickers = ["AAPL", "NESN", "VOO", "MSFT"]
3 shares = [10, 4, 2, 3]
4 price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5 # ... but not in price
6
7 total = 0.0
8 for i in range(len(tickers)):
9     value = shares[i] * price[tickers[i]]
10    total += value
11    if value >= 1500:
12        size = "large"
13    elif value >= 500:
14        size = "medium"
15    else:
16        size = "small"
17    print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19 years = 0
20 grown = total
21 while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24 print(f"total {total:,.2f}, doubles in {years} years")
```

In tickers ...

... but not in price

REMINDER

A crash inside a loop

Read it the session 2 way: last line first, `KeyError: 'MSFT'`. Then the line: **9**. Then the piece: the carets point at `price[tickers[i]]`.

New today: a loop runs line 9 **many** times. Which pass failed? The output **before** the traceback tells you: three positions were printed, so the fourth pass crashed.

The fix belongs in the data, not the loop: add a price for MSFT.

```
$ python ledger.py
```

```
AAPL MSFT ['NESN', 'VOO', 'MSFT'] 102.3
```

```
AAPL 1,905.00 large
```

```
NESN 409.20 small
```

```
VOO 1,024.00 medium
```

Pass 3 worked

```
Traceback (most recent call last):
```

```
File "ledger.py", line 9, in <module>
```

```
value = shares[i] * price[tickers[i]]
```

```
KeyError: 'MSFT'
```

Pass 4 crashed

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  price["NESN"] = 103.0  ← Replace a value
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")

```

```

$ python ledger.py
AAPL  1,905.00  large
NESN   412.00  small  ← 4 × 103.0
VOO    1,024.00  medium
total 3,341.00, doubles in 15 years

```

Adding and changing entries

Assign to a key. **Existing key**: the value is replaced. Line 5 now sets NESN's price to 103.0, and the loop uses the new price.

New key: a new entry. `price["MSFT"] = 410.0` would add MSFT.

Each key appears **once**. A dict cannot hold two prices for NESN.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL   1,905.00  large
NESN    409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

Looping over a dict

A `for` loop over a dict gives you its **keys**, in the order they were added:

```
price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
for ticker in price:
    print(ticker, price[ticker])
```

```
AAPL 190.5
NESN 102.3
VOO 512.0
```

With the key in hand, `price[ticker]` gives the value.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

Tuples: values that belong together

A **tuple** is like a list in round brackets, but it **cannot change**. Use it for a fixed group, such as one position:

```
pos = ("NESN", 4)
ticker, n = pos
print(pos, pos[0], ticker, n, type(pos))
```

```
('NESN', 4) NESN NESN 4 <class 'tuple'>
```

`ticker, n = pos` is **unpacking**: two names on the left take the two values, in order. You will meet tuples mostly this way, in the next slides.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

keys(), values(), items()

```
price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
print(list(price.keys()))
print(list(price.values()))
for ticker, p in price.items():
    print(ticker, p)
```

```
['AAPL', 'NESN', 'VOO']
[190.5, 102.3, 512.0]
AAPL 190.5
NESN 102.3
VOO 512.0
```

`.items()` gives (key, value) tuples. Unpacking them in the loop header gives you both at once: no `price[ticker]` needed.

ledger.py

```
1 # ledger.py - the whole ledger: lists, a dict, loops
2 tickers = ["AAPL", "NESN", "VOO"]
3 shares = [10, 4, 2]
4 price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5 print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7 total = 0.0
8 for i in range(len(tickers)):
9     value = shares[i] * price[tickers[i]]
10    total += value
11    if value >= 1500:
12        size = "large"
13    elif value >= 500:
14        size = "medium"
15    else:
16        size = "small"
17    print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19 years = 0
20 grown = total
21 while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24 print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL 1,905.00 large
NESN 409.20 small
VOO 1,024.00 medium
total 3,338.20, doubles in 15 years
```

+ GOOD TO KNOW

How professionals loop: enumerate and zip

You will see these in almost every real script, and in AI-written code:

tickers	"AAPL"	"NESN"	"VOO"
shares	10	4	2
	↓	↓	↓
zip	("AAPL", 10)	("NESN", 4)	("VOO", 2)

- `for ticker, n in zip(tickers, shares):` walks two lists side by side, like a zipper.
- `for i, ticker in enumerate(tickers):` gives the position **and** the item.

Both hand out tuples, unpacked in the loop header.
Reading them is enough for now.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

The dict's own operations

You write	What it does
<code>d[key]</code>	the value for <code>key</code> ; <code>KeyError</code> if missing
<code>d[key] = v</code>	add a new entry, or replace the value
<code>key in d</code>	<code>True</code> if <code>key</code> is a key
<code>d.get(key, default)</code>	the value, or <code>default</code> if missing
<code>len(d)</code>	the number of entries
<code>for key in d:</code>	go through the keys, in the order they were added

Compare with the list: a dict has no position, no slicing, no `+`. Different type, different operations.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL    1,905.00  large
NESN     409.20  small
VOO     1,024.00  medium
total 3,338.20, doubles in 15 years
```

A list of dicts: a table before pandas

One dict per position, all in a list:

```
ledger = [{"ticker": "AAPL", "shares": 10, "price": 190.5},
          {"ticker": "NESN", "shares": 4, "price": 102.3},
          {"ticker": "VOO", "shares": 2, "price": 512.0}]
print(ledger[1]["price"])
```

102.3

ledger[1] {"ticker": "NESN", ..., "price": 102.3}



...["price"] 102.3

A **row** is a dict, the **table** is a list. Part 6's `DataFrame` stores the same thing, built for speed.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

Three kinds of brackets

Brackets	Right after a name	On their own
()	a call: <code>print(x), len(a)</code>	grouping: <code>(value + cash)</code>
[]	look up: <code>tickers[0], price["NESN"]</code>	a new list: <code>[10, 4, 2]</code>
{ }	—	a new dict: <code>{"AAPL": 190.5}</code>

When you read code, the brackets tell you what is happening before you read the names.

PART 5

while loops

Repeat until a condition stops being true.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:  < Checked before every pass
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

How many passes

while: repeat while the condition is True

Question: at 5% a year, how many years until the ledger is worth twice as much?

- Line 21 checks the condition. `True`: run the body (lines 22–23), then check again.
- `False`: the loop ends, and Python continues at line 24.

Nobody knows in advance how many passes it takes. That is exactly when you use `while`.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:  <grown < 6676.4?
22     grown *= 1.05
23     years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

Tracing the while loop

The target is `2 * total = 6676.4`:

years	grown	grown < 6676.4?
0	3338.2	True : run the body
1	3505.11	True
2	3680.37	True
...	...	...
14	6609.41	True
15	6939.88	False : stop

(`grown` rounded to cents here; the loop keeps full floats.)

ledger.py

```
1 # ledger.py - the whole ledger: lists, a dict, loops
2 tickers = ["AAPL", "NESN", "VOO"]
3 shares = [10, 4, 2]
4 price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5 print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7 total = 0.0
8 for i in range(len(tickers)): Known: 3 positions
9     value = shares[i] * price[tickers[i]]
10    total += value
11    if value >= 1500:
12        size = "large"
13    elif value >= 500:
14        size = "medium"
15    else:
16        size = "small"
17    print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19 years = 0
20 grown = total
21 while grown < 2 * total: Unknown: until doubled
22     grown *= 1.05
23     years += 1
24 print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL 1,905.00 large
NESN 409.20 small
VOO 1,024.00 medium
total 3,338.20, doubles in 15 years
```

for or while?

	for	while
Repeats	once per item	as long as a condition is True
Number of passes	known: the items	unknown in advance
Line in our script	8	21
Stops	by itself	only when the body makes the condition False

Most loops in data work are `for` loops. Reach for `while` when the question is "until when?"

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22     grown *= 1.05  # Changes the condition
23     years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

A while loop that never stops

If the body never makes the condition `False`, the loop runs **forever**:

```
grown = 3338.2
while grown < 6676.4:
    years += 1          # forgot: grown *= 1.05
```

The program hangs. No error, no output. Stop it with **Ctrl+C** in the terminal, or the stop button (■) in VS Code.

Before running a `while` loop, find the line in the body that moves the condition towards `False`. Line 22 is ours.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

break: leave a loop early

```
tickers = ["AAPL", "NESN", "VOO"]
for ticker in tickers:
    print("checking", ticker)
    if ticker == "NESN":
        print("found it")
        break
```

```
checking AAPL
checking NESN
found it
```

break ends the loop at once, even with items left. VOO is never checked.

Useful for searching: stop as soon as you have found what you need.

PART 6

Libraries bring their own types

numpy and pandas: the types data science is
built on.

ledger_np.py

```
1  # ledger_np.py - the same ledger, with numpy and pandas
2  import numpy as np  # Load a library (session 4)
3  import pandas as pd
4
5  shares = np.array([10, 4, 2])
6  prices = np.array([190.5, 102.3, 512.0])
7  values = shares * prices
8  print(values, values.sum())
9  print([10, 4, 2] * 2, shares * 2)
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "VOO"],
12                        "shares": [10, 4, 2],
13                        "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger)
16 print(type(shares), type(ledger))
```

```
$ python ledger_np.py
[1905.   409.2 1024. ] 3338.2
[10, 4, 2, 10, 4, 2] [20  8  4]
   ticker  shares  price  value
0  AAPL      10   190.5  1905.0
1  NESN       4   102.3   409.2
2  VOO        2   512.0  1024.0
<class 'numpy.ndarray'> <class 'pandas.DataFrame'>
```

A library is code someone else wrote

Session 1: Python's strength is its libraries. Many of them bring **new types**, each with its own operations.

On the left, the same ledger, with two libraries:

- **numpy**: fast arrays of numbers.
- **pandas**: tables with named columns.

`import numpy as np` loads the library under a short name. Session 4 explains `import`; today, just read it as "use numpy".

ledger_np.py

```
1  # ledger_np.py – the same ledger, with numpy and pandas
2  import numpy as np
3  import pandas as pd
4
5  shares = np.array([10, 4, 2])
6  prices = np.array([190.5, 102.3, 512.0])
7  values = shares * prices  No loop!
8  print(values, values.sum())
9  print([10, 4, 2] * 2, shares * 2)
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "VOO"],
12                        "shares": [10, 4, 2],
13                        "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger)
16 print(type(shares), type(ledger))
```

numpy arrays: one operation, every element

shares	10	4	2
	×	×	×
prices	190.5	102.3	512.0
	↓	↓	↓
values	1905.0	409.2	1024.0

An array's `*` works **element by element**: position 0 with position 0, and so on. Lines 7–10 of `ledger.py` in one line, with no loop.

`values.sum()` is a method of the array type: `3338.2`.

```
$ python ledger_np.py
```

```
[1905.   409.2 1024. ] 3338.2 values.sum()
[10, 4, 2, 10, 4, 2] [20  8  4]
   ticker  shares  price  value
0   AAPL      10  190.5  1905.0
1   NESN       4  102.3   409.2
2    VOO       2  512.0  1024.0
<class 'numpy.ndarray'> <class 'pandas.DataFrame'>
```

```

ledger_np.py
1  # ledger_np.py – the same ledger, with numpy and pandas
2  import numpy as np
3  import pandas as pd
4
5  shares = np.array([10, 4, 2])
6  prices = np.array([190.5, 102.3, 512.0])
7  values = shares * prices
8  print(values, values.sum())
9  print([10, 4, 2] * 2, shares * 2)
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "VOO"],
12                          "shares": [10, 4, 2],
13                          "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger)
16 print(type(shares), type(ledger))

```

```

$ python ledger_np.py
[1905.  409.2 1024. ] 3338.2
[10, 4, 2, 10, 4, 2] [20  8  4]
  ticker  shares  price  value
0  AAPL      10  190.5  1905.0
1  NESN       4  102.3   409.2
2   VOO       2  512.0  1024.0
<class 'numpy.ndarray'> <class 'pandas.DataFrame'>

```

Same symbol, four types, four operations

You write	Type	You get	What * means
3 * 2	int	6	multiply
"3" * 2	str	"33"	repeat the text
[10, 4, 2] * 2	list	[10, 4, 2, 10, 4, 2]	repeat the list
shares * 2	numpy array	[20 8 4]	multiply each element

[10, 4, 2] and np.array([10, 4, 2]) **look** almost the same. Their * does something completely different. Before you read an operator, find out the type.

```

1  # ledger_np.py – the same ledger, with numpy and pandas
2  import numpy as np
3  import pandas as pd
4
5  shares = np.array([10, 4, 2])  ← All int
6  prices = np.array([190.5, 102.3, 512.0])
7  values = shares *  ← All float
8  print(values, values.sum())
9  print(np.array([10, 4, 2.5]), np.array([10, "NESN"]))
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "VOO"],
12                          "shares": [10, 4, 2],
13                          "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger)
16 print(type(shares), type(ledger))

```

An array holds one type

A list can mix types. A numpy array cannot: it converts every element to **one** type. Line 9 now builds two mixed arrays.

First array: the ints became floats, like session 2's ladder. Second array: the `10` became the **text** `'10'`. numpy coerces even to text, which plain Python never does. One more reason to check the type of your data.

```

$ python ledger_np.py
[1905.   409.2 1024. ] 3338.2
[10.   4.   2.5] ['10' 'NESN']
   ticker  shares  price  value
0  AAPL      10  190.5  1905.0
1  NESN       4  102.3   409.2
2   VOO       2  512.0  1024.0
<class 'numpy.ndarray'> <class 'pandas.DataFrame'>

```

```

1  # ledger_np.py – the same ledger, with numpy and pandas
2  import numpy as np
3  import pandas as pd
4
5  shares = np.array([10, 4, 2])
6  prices = np.array([190.5, 102.3, 512.0])
7  values = shares * prices
8  print(values, values.sum())
9  print(values > 1000, values[values > 1000])
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "VOO"],
12                          "shares": [10, 4, 2],
13                          "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger)
16 print(type(shares), type(ledger))

```

One comparison per element

```

$ python ledger_np.py
[1905.   409.2 1024. ] 3338.2
[ True False  True] [1905. 1024.]
   ticker  shares  price  value
0    AAPL      10   190.5  1905.0
1    NESN       4   102.3   409.2
2     VOO       2   512.0  1024.0
<class 'numpy.ndarray'> <class 'pandas.DataFrame'>

```

Comparisons work element by element too

Line 9 now compares the whole array with 1000, then keeps the matches:

values	1905.0	409.2	1024.0
> 1000	True	False	True
values[...]	1905.0	dropped	1024.0

The "collect" pattern from Part 2, with no loop:
compare, then keep what is `True`.

```

ledger_np.py
1  # ledger_np.py – the same ledger, with numpy and pandas
2  import numpy as np
3  import pandas as pd
4
5  shares = np.array([10, 4, 2])
6  prices = np.array([190.5, 102.3, 512.0])
7  values = shares * prices
8  print(values, values.sum())
9  print([10, 4, 2] * 2, shares * 2)
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "VOO"],
12                        "shares": [10, 4, 2],
13                        "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger)
16 print(type(shares), type(ledger))

```

```

$ python ledger_np.py
[1905.   409.2 1024. ] 3338.2
[10, 4, 2, 10, 4, 2] [20  8  4]
  ticker  shares  price  value
0  AAPL      10  190.5  1905.0
1  NESN       4  102.3   409.2
2   VOO       2  512.0  1024.0
<class 'numpy.ndarray'> <class 'pandas.DataFrame'>

```

Named columns

pandas DataFrame: a table

A `DataFrame` is a table: **columns with names**, rows numbered from 0.

- A column is looked up by name, like a dict key: `ledger["price"]`.
- Column arithmetic works element by element, like numpy (line 14).
- Assigning to a new name adds a column, like adding a dict key.

Lists, dicts, element-by-element arithmetic: today's ideas, combined into one type. Part 2 of the course is built on it.

ledger_np.py

```
1  # ledger_np.py – the same ledger, with numpy and pandas
2  import numpy as np
3  import pandas as pd
4
5  shares = np.array([10, 4, 2])
6  prices = np.array([190.5, 102.3, 512.0])
7  values = shares * prices
8  print(values, values.sum())
9  print([10, 4, 2] * 2, shares * 2)
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "VOO"],
12                          "shares": [10, 4, 2],
13                          "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger)
16 print(type(shares), type(ledger))
```

```
$ python ledger_np.py
```

```
[1905.   409.2 1024. ] 3338.2
[10, 4, 2, 10, 4, 2] [20  8  4]
   ticker  shares  price  value
0   AAPL      10   190.5  1905.0
1   NESN       4   102.3   409.2
2    VOO       2   512.0  1024.0
```

numpy's array type

```
<class 'numpy.ndarray'> <class 'pandas.DataFrame'>
```

What type is it? Ask Python

`type()` works on library values too: `shares` is a `numpy.ndarray`, and `ledger` is a `pandas.DataFrame`.

The library name is part of the type's name: it tells you **whose operations** you are using. When `*` or `+` surprises you, `type()` is the first question.

Older pandas versions print the longer name `pandas.core.frame.DataFrame`. Same type.

ledger_np.py

```
1  # ledger_np.py – the same ledger, with numpy and pandas
2  import numpy as np
3  import pandas as pd
4
5  shares = np.array([10, 4, 2])
6  prices = np.array([190.5, 102.3, 512.0])
7  values = shares * prices
8  print(values, values.sum())
9  print([10, 4, 2] * 2, shares * 2)
10
11 ledger = pd.DataFrame({"ticker": ["AAPL", "NESN", "VOO"],
12                        "shares": [10, 4, 2],
13                        "price": [190.5, 102.3, 512.0]})
14 ledger["value"] = ledger["shares"] * ledger["price"]
15 print(ledger)
16 print(type(shares), type(ledger))
```

```
$ python ledger_np.py
```

```
[1905.   409.2 1024. ] 3338.2
[10, 4, 2, 10, 4, 2] [20  8  4]
   ticker  shares  price  value
0   AAPL      10  190.5  1905.0
1   NESN       4  102.3   409.2
2    VOO       2  512.0  1024.0
<class 'numpy.ndarray'> <class 'pandas.DataFrame'>
```

+ GOOD TO KNOW

Why not just use lists?

- **Speed and size:** numpy stores numbers compactly and works on whole arrays at once. It is made for millions of values.
- **Shorter code:** `shares * prices` replaces a loop. Less code, fewer places for a bug.
- **Tools:** pandas reads CSV and Excel files, groups, filters and plots.

The price: new types with their own rules. `+` on a list joins; `+` on an array adds. You now know to check.

```

1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  for i in range(len(tickers)):
8      total = 0.0  < Moved inside the loop
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22     grown *= 1.05
23     years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")

```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
```

```
AAPL 1,905.00 large
```

```
NESN 409.20 small
```

```
VOO 1,024.00 medium
```

```
total 1,024.00, doubles in 15 years < Is this right?
```

Spot the lie

A classmate swapped lines 7 and 8: `total = 0.0` now sits **inside** the loop.

It runs. Every line prints. The total looks like money.

Is it right? Which kind of failure is this, and how would you catch it?

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0  ← Before the loop: once
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

← The right answer

The lie, explained

`total = 0.0` is **inside** the loop, so it runs on every pass. Each pass throws the running total away. Only the last position survives: VOO, 1,024.00.

Catch it with a rough check: the total must be at least as big as the largest position. AAPL alone is worth 1,905.

The fix: start the accumulator **before** the loop, as line 7 does. Accumulator pattern, step 1.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:.,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL    1,905.00  large
NESN     409.20  small
VOO     1,024.00  medium
total 3,338.20, doubles in 15 years
```

Collections at a glance

Type	Example	Look up by	Can change?	+ me
str	"NESN"	position	no	join
list	[10, 4, 2]	position	yes	join
tuple	("NESN", 4)	position	no	join
dict	{"NESN": 102.3}	key	yes	TypeEr
numpy array	np.array([10, 4, 2])	position	yes	add ea elemer
pandas DataFrame	the ledger table	column name	yes	columr columr

Six types, one question to ask of each: **what do its operations mean?**

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO   1,024.00  medium
total 3,338.20, doubles in 15 years
```

Read it again

The same 24 lines as at the start. Every line is now yours:

- the **types** on lines 2–4: two lists and a dict,
- **indexing and slicing** on line 5,
- the **for** loop and its **accumulator**, lines 7–10,
- the **if / elif / else** chain, lines 11–16,
- the **while** loop, lines 19–23.

ledger.py

```
1  # ledger.py - the whole ledger: lists, a dict, loops
2  tickers = ["AAPL", "NESN", "VOO"]
3  shares = [10, 4, 2]
4  price = {"AAPL": 190.5, "NESN": 102.3, "VOO": 512.0}
5  print(tickers[0], tickers[-1], tickers[1:], price["NESN"])
6
7  total = 0.0
8  for i in range(len(tickers)):
9      value = shares[i] * price[tickers[i]]
10     total += value
11     if value >= 1500:
12         size = "large"
13     elif value >= 500:
14         size = "medium"
15     else:
16         size = "small"
17     print(f"{tickers[i]:<5} {value:>9,.2f} {size}")
18
19  years = 0
20  grown = total
21  while grown < 2 * total:
22      grown *= 1.05
23      years += 1
24  print(f"total {total:,.2f}, doubles in {years} years")
```

```
$ python ledger.py
```

```
AAPL VOO ['NESN', 'VOO'] 102.3
AAPL  1,905.00  large
NESN   409.20  small
VOO    1,024.00  medium
total 3,338.20, doubles in 15 years
```

Check yourself

On paper first, then check with your neighbour:

1. What is `tickers[1:3]`? And `shares[-1]`?
2. After `shares.append(5)`, what is `len(shares)`?
3. A position worth exactly `500.0`: which size does line 11–16 give it?
4. What is `[1, 2] + [3]`? And `np.array([1, 2]) + np.array([3, 4])`?
5. How many times does the body of the `while` loop run?

What you can now do

- Build lists and dicts, and look values up by position or by key.
- Index from 0 and from -1; slice with `[start:stop]`, stop not included.
- Repeat work with `for`, and use the accumulator, counting and collecting patterns.
- Choose with `if` / `elif` / `else`, biggest threshold first.
- Repeat until done with `while`, and make sure it stops.
- Say what `*` and `+` mean for `int`, `str`, `list` and a numpy array.

Next session: **Monday 5 October, functions and imports**. The ledger script gets organised into functions.